

OPL Cloud 白皮书

让复杂知识工作跨越本机、私有数据、远端资源与对外服务，仍然保持连续、受控、可复查

云端应让用户持续专注成果，并把基础设施留在系统背后。OPL Cloud 让工作从本机自然延伸到在线 Workspace、私有数据、高性能计算、协作和对外 Agent 服务，同时把权限、成本、证据与专业判断放在各自正确的责任边界。

让复杂知识工作跨越本机、私有数据、远端资源与对外服务，仍然保持连续、受控、可复查

目录

1 OPL Cloud 白皮书 2

2 定位摘要 2

3 为什么复杂知识工作需要不同的云 2

4 OPL Cloud 的答案：连续工作面，分层责任 4

5 OPL Cloud 的五大设计原则 4

5.1 一、成果优先于基础设施 4

5.2 二、本机与云端是同一条工作链 4

5.3 三、先计划，再授予资源权力 5

5.4 四、管理面与工作面各司其职 5

5.5 五、证据连接全程，专业结论归领域 owner 5

6 OPL Cloud 的能力版图 5

6.1 从产品到具体实例 6

7 从 Agent Package 到对外服务 6

8 标准任务生命周期 6

8.1 1. 从本机形成问题 7

8.2 2. 选择已验证的专业能力 7

8.3 3. 形成资源计划 7

8.4 4. 在关键边界批准 7

8.5 5. 在数据所在处执行 7

8.6 6. 把结果带回工作台 7

8.7 7. 交给领域独立审阅 7

8.8 8. 留下可继续的回执 7

9 四类用户路径 7

10 系统模型 8

11 专业工作流 8

12 六类信任机制 8

12.1 范围可信 8

12.2 执行可信 8

12.3 版本可信 9

12.4 服务可信 9

12.5 结果可信 9

12.6 接力可信 9

13 本文边界 9

14 结语 9

1 OPL Cloud 白皮书

让复杂知识工作跨越本机、私有数据、远端资源与对外服务，仍然保持连续、受控、可复查

发布日期：2026-07-15 最近修订：2026-08-11

适用对象：正在使用 AI 推进科研、基金、汇报、书籍或其他长期知识项目，希望沿用现有工作方式使用在线工作台、协作资源与远端计算，或者把自己开发的 OPL Agent 提供给外部用户的个人、实验室和机构。

核心判断：云端应让用户持续专注成果，并把基础设施留在系统背后。OPL Cloud 让工作从本机自然延伸到在线 Workspace、私有数据、高性能计算、协作和对外 Agent 服务，同时把权限、成本、证据与专业判断放在各自正确的责任边界。

2 定位摘要

OPL Cloud 是 One Person Lab 面向复杂知识工作的云端产品架构：用户继续围绕问题、材料、阶段、产物和审阅推进工作；系统在背后连接 AI、在线工作空间、账号策略、远端资源、Agent 服务和证据回执。

本仓库承担 **OPL Cloud 目标产品架构与实现族导航**。本文解释为什么采用这样的设计、各部分如何协作，以及这种协作怎样改善用户体验。目标架构与当前交付分别说明；具体能力是否可用、运行到什么程度、如何计费 and 何时发布，以对应实现仓、机器合同、运行读回与正式公告为准。

OPL Cloud 希望兑现五个用户结果：

- 从本机开始，需要时再接入在线工作台或远端资源，原项目直接延续。
- 私有数据可以留在机构边界内，计算去到数据所在的位置。
- 账号或协作成员在关键动作前完成授权，日常工作保持连续。
- Agent 开发者可以把经过验证的精确版本发布为 API、嵌入组件或托管界面，并直接获得统一服务栈。
- 结果回到原工作链，并带有足够的来源、环境、审阅和继续线索。

3 为什么复杂知识工作需要不同的云

一次普通云任务通常可以用“上传、运行、下载”描述。长期知识工作还包含持续修改、授权、审阅、交接和专业判断。

研究者可能先在本机整理问题和材料，随后发现分析需要医院内网数据、实验室高性能计算集群和特定软件环境。协作成员需要知道谁可以访问数据、任务会消耗多少资源、输出应回到哪里；领域负责人还要判断结果是否足以支持论文中的结论。Agent 开发者还可能希望把成熟能力提供给外部 App 或网站。数周后，另一位成员需要理解这次分析从哪里来，并接着修改。

如果每遇到一个问题就切换到一套独立系统，用户会同时维护本地文件、云盘、作业队列、模型账号、审批消息、环境说明和人工日志。核心困难在于让这些工具围绕同一个成果协作。

OPL Cloud 因此以五组用户矛盾为设计起点：

用户需要	常见代价	OPL Cloud 的设计选择
使用更强的 AI 与计算	被迫离开当前工作台	本机 App 与在线 Workspace 使用连续的工作模型
使用私有数据与机构资源	复制数据或暴露凭据	让 Fabric 把执行绑定到数据所在资源
账号治理权限与成本	每一步都经过管理后台	Console 只在账号策略和批准边界介入
把 Agent 提供给外部用户	每个开发者重复搭建 API、鉴权、运行环境和前端	OPL Serve 发布精确 Revision，并提供统一 API 与可选 UI 模板
长期解释和复查结果	依赖成员记忆与手工记录	Ledger 保存引用与回执，专业结论仍由领域 owner 判断

这五个选择共同指向一个原则：让工作保持连续，让权力保持分离。

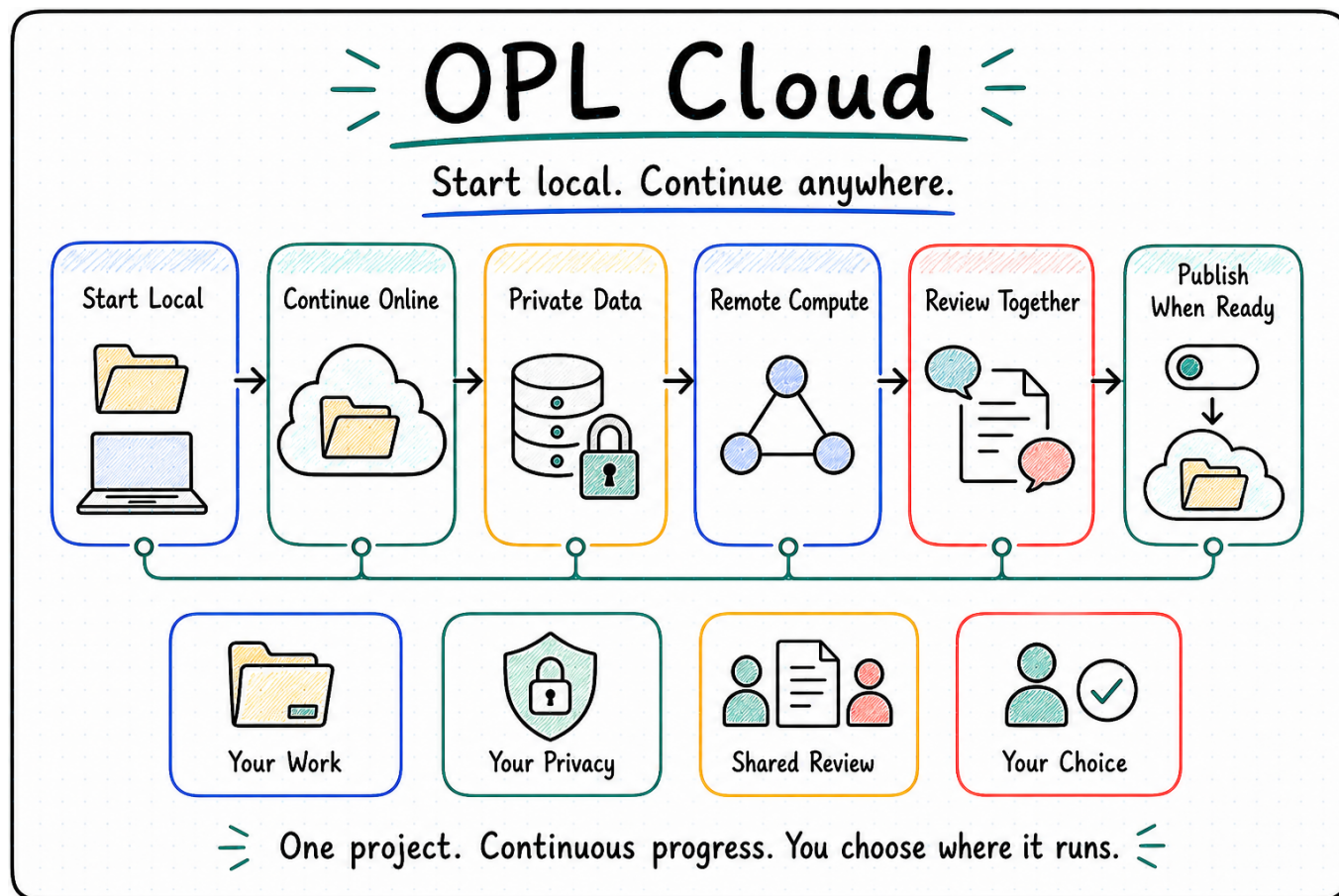


图 1: OPL Cloud 让同一项目从本机自然延伸到在线协作、私有数据、远端计算与按需发布。

4 OPL Cloud 的答案：连续工作面，分层责任

用户真正面对的是 OPL App 或 OPL Workspace。两者都围绕项目、材料、任务、产物、审阅和下一步组织体验。Cloud 能力在需要时自然融入这条工作链。

用户工作面	OPL App / OPL Workspace
AI 能力	OPL Gateway
Agent 服务发布	OPL Serve
资源连接与执行	OPL Fabric
账号政策与批准	OPL Console
来源与运行回执	OPL Ledger
专业质量判断	MAS / MAG / RCA 等领域 Agent
能力包 identity/发布	Package owner
能力包物理生命周期	配置的原生 carrier
能力包发现与状态聚合	OPL Packages (由 OPL Framework 持有)
调用与会话执行	OPL Runway (由 OPL Framework 持有)

分层为每项权力指定清楚的责任方：

- Gateway 让模型接入、路由和用量有稳定入口。
- Serve 把精确 Agent Revision 发布为 API、嵌入组件或托管界面，并持有服务、部署、端点与流量状态。
- Fabric 通过 provider-neutral 能力接口连接计算、存储、环境、数据源与外部系统，并把一次任务绑定到获准资源；腾讯云只是第一条适配路径。
- Console 是用户与管理员的统一中台，负责账号、Workspace 集合、余额与费用投影、协作角色、额度、策略和批准；日常专业工作留在 App 或 Workspace，任务执行由 Runway 与 Fabric 承载，服务运行真相由 Serve 持有。
- Ledger 记录计划、批准、输入引用、环境、输出引用、审阅结果和继续入口，并引用领域 owner source。
- 领域 Agent 保留专业策略、证据判断、质量结论与交付权威。
- Package owner 持有稳定 identity、capabilities、entrypoints 与精确发布 revision；配置的原生 carrier 持有安装、更新、移除与 fresh installed/callable readback；OPL Packages 只负责发现、carrier 委托与状态聚合。Cloud 各面共享这些 owner/carrier refs。
- OPL Runway 统一持有 Invocation、Session 和执行 provider 生命周期；sandbox 或外部 provider id 作为执行引用，Invocation 与 Session 事实始终来自 Runway。

因此，一个用户可以只理解“我要完成什么、系统准备使用什么、结果在哪里”。模块边界留在系统背后，为这种简单体验提供可靠基础。

5 OPL Cloud 的五大设计原则

5.1 一、成果优先于基础设施

用户首先看到问题、材料、阶段、产物和下一步。容器、队列、存储桶或内部服务名只在资源选择影响权限、成本或复查时进入界面。

这使复杂计算仍然表现为工作的一部分：用户批准“为这项分析使用实验室集群和指定环境”，收到图表、报告和审阅结果；作业编号留在诊断层。

5.2 二、本机与云端是同一条工作链

本机 OPL App 适合个人控制、敏感材料和日常编辑；在线 OPL Workspace 适合远程访问、协作和托管执行。两者是同一工作模型的两种载体。一个用户账号可以拥有零个或多个相互独立的 Workspace；每个 Workspace 本质上是一套带 WebUI 的 OPL App 容器部署，并拥有独立 identity、URL、runtime、资源绑定、账期、凭据与回执。产品层不设固定数量上限，每次创建仍受余额、provider 容量、额度与策略约束。

用户可以从本机开始，在需要在线访问或更强资源时接入 Cloud，创建或选择合适的 Workspace，再把结果带回原项目。环境改变后，任务身份、材料引用、产物关系和继续入口完整保留。一个账号可以发布多个 Agent Service；Service 属于对外部署资源，与 Workspace 采用不同的身份边界。

5.3 三、先计划，再授予资源权力

正式远端任务先形成资源计划：要访问哪些输入、使用什么环境与计算、写入什么位置、预计消耗什么、如何停止。个人、账号或机构只对可理解的计划授权。

批准把危险动作和普通工作分开。低风险读取可以按既定策略直接进行；敏感数据出域、高成本计算、共享连接器、服务公开和纳管资源变更则在明确边界上停下来。

5.4 四、管理面与工作面各司其职

Console 服务账号所有者与管理员：协作角色、权限、预算、额度、策略和批准。研究者、作者和设计者仍在 App 或 Workspace 中完成工作。

Gateway 持有唯一可消费余额和 AI 实时用量；Console 持有账号总费用视图、定价与结算策略，并按 Workspace、按账期发起月费扣款；Fabric 只返回资源与 provider 事实，余额为零；Ledger 保存收费、退款和资源对账回执。这样既能在 Console 看到一份总账，又不会产生第二个可变余额真相。

同样，能力包的 identity 与精确发布 revision 归 Package owner，物理安装、更新、移除和 repair 归配置的原生 carrier，Framework 只委托并聚合 fresh readback；Console 决定账号允许使用或发布哪些 owner revision；Serve 持有 Agent Service、Revision、Deployment 和流量状态；Runway 持有 Invocation 与 Session 生命周期；Fabric 为一次执行绑定所需资源。这样的分层 owner 设计，让“发布了什么”“本机实际装了什么”和“当前服务是什么”各有唯一来源。

5.5 五、证据连接全程，专业结论归领域 owner

Ledger 让计划、批准、环境、输入、输出和审阅彼此可追溯。记录负责说明来路，科研结论由 MAS 等领域 Agent 和人类负责人判断；基金、视觉交付和书稿也各有自己的质量边界。

OPL Cloud 负责让判断有来路、结果能复查、工作可接力。运行成功、文件存在和回执齐全作为过程证据，专业 ready 由领域 owner 给出。

6 OPL Cloud 的能力版图

这张能力版图按用户在一条工作链中遇到的八个问题组织。每个问题只有一个主要责任方。

用户问题	主要责任方	用户获得什么
我在哪里继续工作？	OPL App / Workspace	连续的项目、任务、产物与审阅体验
AI 从哪里来、用了多少？	OPL Gateway	稳定模型入口、路由与用量信号
怎样把 Agent 提供给外部用户？	OPL Serve	精确 Revision、API、嵌入组件、托管 UI 与服务治理
数据、工具和计算怎样接入？	OPL Fabric	可说明来源、环境与输出位置的资源绑定
账号允许谁使用或发布什么？	OPL Console	账号策略、批准、额度与管理视图
Agent 和能力包发布了什么、本机实际可调用什么？	Package owner + 原生 carrier	精确发布 revision 与 fresh installed/callable readback
这次运行发生了什么？	OPL Ledger	引用、回执、审阅结果与继续入口
结果在专业上是否成立？	领域 Agent / 人类 owner	领域质量判断、修订意见与交付决定

这里最重要的是 **不把不同状态轴合并成第二份 registry truth**。Package owner descriptor 与发布 revision 定义“这是什么”，原生 carrier readback 定义“这里实际装了什么、能否调用”，Framework 只聚合并委托。Console 投影账号可用性，Fabric 读取执行所需绑定，Workspace 展示聚合状态，Serve 创建引用 owner 精确发布 revision 的不可变 Revision，Ledger 引用相关回执。

同理，Ledger 连接研究数据引用，Serve 与 Fabric 持有各自状态变更，领域 Agent 判断专业结果，Workspace 展示来自 Serve 的服务状态。Hosted UI 与嵌入组件统一调用公开 Serve API。用户看到一致体验，背后的权威各归其主。

6.1 从产品到具体实例

OPL Cloud 产品与某次安装不是同一个对象。one-person-lab-cloud 同时持有产品架构、白皮书以及可复用的 Console、Control Plane、Fabric 与 Ledger 实现；opl-cloud 只作为内部 package、image、service 与 namespace 标识。具体安装由实例配置物化。第一期商业实例 medopl 由 opl-instance-medopl 持有域名、provider profile、启用套餐与价格、镜像 pin、secret 引用和部署回执，不复制产品或 runtime 代码。

一个实例可以安装在公有云、本地服务器或 Mac 上。实例选择获准的 Fabric provider profile；medopl 首先使用 tencent-tke，本地安装可以使用 local-docker，自托管集群可以使用通用 kubernetes。Launch 与 Recovery 共享一套产品状态机，provider 特有事实、重试和恢复留在对应 adapter 内部。

7 从 Agent Package 到对外服务

OMA 或其他兼容方式可以帮助用户设计 Agent，对外服务从 Package owner 的精确发布 revision 出发。开发来源、Package、Service 与一次调用是不同对象：Package 说明“这是什么能力和发布 revision”，Service 说明“以什么政策提供给谁”，Invocation 或 Session 说明“这一次实际发生了什么”。

```
Agent 设计
-> Agent Package candidate
-> owner descriptor + exact publication revision
-> Service Entrypoint Contract
-> OPL Serve Agent Service
-> immutable Agent Revision
-> Deployment + traffic policy
-> API / Embed / Hosted UI
-> Invocation / Session
-> Ledger receipt refs
```

OPL Serve 提供三种交付方式，并让它们共享一套服务合同：

方式	面向谁	设计边界
API	已有 App、网站或后端	服务端凭据、幂等调用、异步状态、事件流和 Webhook
Embed	已有网站中的交互组件	使用短期浏览器凭据，保护发布者与 provider 密钥
Hosted UI	需要现成前端的发布者	Task、Report、Workflow、Chat 模板调用同一公开 API

公网流量先进入 Serve Agent Edge，由它处理 consumer 身份、鉴权、输入校验、额度、限流、路由、事件流和签名 Webhook，再由 Runway 创建 Invocation 或 Session。Fabric 负责 sandbox、计算、存储、网络、secret 注入和产物收集；Gateway 提供模型访问；Connect 连接外部数据与工具；Ledger 保存引用与回执。稳定公网入口统一由 Serve Agent Edge 提供。

Hosted UI 可以投影发布者名称、Logo、主题、域名、输入输出 schema、文件上传、进度、产物与支持链接，并聚焦 Agent 交互模板；服务状态由 Serve 持有。发布者可以拥有多个 Agent Service 和多个独立 Workspace，两者仍是不同的产品身份与生命周期。

第一阶段聚焦 Agent Service 的安全发布与运行，由 OPL 向 Agent 发布者账号归集服务、模型、执行、存储和纳管连接器用量。面向最终消费者的 Marketplace、代收款、税务、退款、KYC、分成和订阅管理属于后续商业产品。

8 标准任务生命周期

下面是一条典型的实验室工作链。它比模块目录更能说明 OPL Cloud 为什么这样设计。

8.1 1. 从本机形成问题

研究者在 OPL App 中整理研究问题、分析计划和已有材料。患者级数据仍留在医院或机构存储，本机项目只保留允许使用的引用和上下文。

8.2 2. 选择已验证的专业能力

项目需要 MAS 及特定统计能力包。Framework 读取 owner descriptor 并聚合配置 carrier 的 fresh installed/callable 状态；required dependency 只检查 identity presence 与所需入口可调用。账号策略可以限制允许的 owner publication refs，Console 不创建另一份 Package 或 installed truth。

8.3 3. 形成资源计划

任务发现本机算力不足，于是形成一份可读计划：在实验室 HPC 上运行、访问某个私有数据引用、使用指定 R/Python 环境、把脱敏结果写回项目输出区，并给出预算、停止方式和预期产物。

8.4 4. 在关键边界批准

Console 根据账号与机构策略检查成员权限、HPC 配额、数据边界和预算。策略已覆盖的动作自动通过；涉及敏感数据或高成本资源时，由有权的人确认。

8.5 5. 在数据所在处执行

Fabric 绑定 HPC、私有存储和软件环境，提交并监控任务。数据默认留在资源所在位置；凭据和原始数据继续由资源 owner 持有。

8.6 6. 把结果带回工作台

计算完成后，图表、汇总和日志引用回到原项目。用户仍在熟悉的 App 或 Workspace 中查看结果、比较版本和决定下一步，基础设施页面保留为诊断与运维入口。

8.7 7. 交给领域独立审阅

MAS 对方法、数字、图表与 claim 的对应关系进行专业审阅。运行成功证明执行完成，研究结论仍由专业审阅确认；发现问题时，修订意见回到同一工作链，必要时再次计划和执行。

8.8 8. 留下可继续的回执

Ledger 连接资源计划、批准、环境、输入引用、输出引用、审阅结果、负责人和继续入口。几个月后，协作成员可以从回执找到相关 owner source，沿完整上下文继续工作。

本机问题

- > 已验证能力包
- > 资源计划
- > 账号或机构批准
- > 私有数据旁执行
- > 结果回到工作台
- > 领域独立审阅
- > 回执与下一轮继续

9 四类用户路径

同一设计可以按需要逐步展开，每个人都从自己所需的 Cloud 层级开始。

个人用户 可以始终以本机 App 为主，只在需要在线访问、Gateway 或远端计算时接入 Cloud。私有项目保持原有材料位置。

Agent 开发者 可以在 OMA 或其他兼容方式帮助下形成 Agent Package，由 Package owner 发布精确 revision 后绑定到 OPL Serve。已有产品调用 API；需要现成前端时选择 Hosted UI 模板。开发者仍对 Agent 的专业承诺、内容和终端客户关系负责。

实验室与协作团队 可以共享批准过的模型、能力包、连接器、环境和计算资源。成员在各自工作面推进项目，管理员只处理账号或机构策略，领域负责人继续对专业结果负责。

企业与机构 可以让内部数据库、私有存储、工具 API、HPC 或 GPU 保持原有 owner，同时通过 Fabric 接入标准执行，通过 Console 管理权限与预算，通过 Ledger 获得可复查引用。

四条路径共享同一组设计判断：从成果出发、按需扩展、关键动作授权、权威各归其主。

10 系统模型

从用户视角看，OPL Cloud 只有四个时刻需要被感知：需要额外能力时、需要发布服务时、需要批准时、需要复查时。

时刻	用户看到	系统保证
需要额外能力	可理解的模型、环境或资源选择	Package owner、原生 carrier、Framework 聚合、Gateway 与 Fabric 各自提供 owner refs
需要发布服务	API、嵌入组件或托管 UI，精确 Revision 与流量状态	Serve 持有服务状态，Runway 持有调用/会话执行
需要批准	访问范围、成本、输出位置和停止方式	Console 执行账号策略，执行归 Runway 与 Fabric
需要复查	产物、审阅结果、来源和继续入口	Ledger 连接 refs，领域 owner 保留结论权威

这种模型把复杂度放在系统内部，同时保持自动化可解释。用户只需关注系统准备做什么、谁批准、结果来自哪里、下一步由谁负责。

11 专业 workflow

科研只是其中一个例子。基金申请需要证据、版本、预算字段和模拟评审；汇报需要材料、故事线、图表、视觉审阅和导出；书稿需要章节、来源、风格与出版交接。这些工作共享云端资源和协作能力，质量结论仍按领域分别给出。

OPL Cloud 因此只上收通用部分：工作面连续性、AI 接入、服务发布、资源计划、权限批准、执行绑定、引用回执和继续入口。领域 Agent 保留专业方法、术语、质量标准、审阅行为和交付权威。

这种“通用基础设施 + 领域判断”的结构有两个好处：用户在不同专业任务中获得一致体验；专业质量也在统一界面下保持完整。

12 六类信任机制

12.1 范围可信

用户知道材料在哪里、任务准备访问什么、是否发生外部传输。敏感数据默认留在用户工作空间、机构存储或私有桶，Cloud 只保存必要的引用、策略和回执。

12.2 执行可信

远端任务说明计算位置、软件环境、输入引用、输出位置和停止方式。Fabric 的职责是忠实执行获准计划，并返回可检查的运行结果。

12.3 版本可信

Package owner 用 descriptor 与精确发布 revision 表达 identity 和 publication provenance；配置的原生 carrier 用 fresh readback 表达安装、更新、移除和 callability；Framework 只聚合两者。App、Workspace、Console 与 Fabric 不复制这些 owner state。

12.4 服务可信

OPL Serve 用 Agent Service、不可变 Revision、Deployment 和流量策略表达对外服务。公网请求先经过 Agent Edge，再由 Runway 绑定精确版本和执行 provider；稳定公网入口统一经 Agent Edge 治理。API、嵌入组件和 Hosted UI 使用同一调用、额度、事件与回执合同。

12.5 结果可信

Ledger 把运行证据交给领域审阅。最终质量、发布、提交或交付决定来自对应领域 owner 和必要的人类 gate。

12.6 接力可信

重要结果带有负责人、输入输出 refs、审阅结果和继续入口。协作成员未来可以回到 owner source 重新理解和修订，始终保有完整来源。

13 本文边界

这份白皮书面向用户解释 OPL Cloud 的设计理念与目标产品边界，回答“为什么这样设计会更好用”。说明书、服务目录、发布公告和 readiness dashboard 分别由对应产品表面提供。

想确认的事实	应读取的权威来源
OPL Cloud 为什么存在、目标体验是什么	本白皮书
某个 Cloud 服务当前实现了什么	对应实现仓、接口合同与测试
某项能力此刻是否可用	运行读回、健康状态与 owner receipt
价格、套餐、安全与发布时间	正式产品页面、服务合同与公告
某份专业成果是否合格	MAS、MAG、RCA 等领域 owner 与人类负责人

当前 OPL Cloud 实现族正在按这些边界演进。“进入目标架构”“合同已经存在”和“测试通过”分别证明方向、实现与验证进展；全部服务交付则以运行读回、发布证据和 owner receipt 为准。

14 结语

好的云端产品让用户继续思考问题、审阅结果和推进成果，同时需要在需要时获得更强 AI、在线工作空间、私有数据连接、远端计算、协作和对外 Agent 服务。

OPL Cloud 选择用连续工作面承接用户，用单一 owner 管理能力包，用 Serve 发布精确 Agent Revision，用 Runway 维持调用与会话生命周期，用 Fabric 连接资源，用 Console 克制地治理账号策略，用 Ledger 保存可追溯引用，再把专业判断交还领域 Agent 与人类负责人。

这套设计的价值来自清楚责任：工作可以跨环境继续，数据始终保持边界，管理服务创作，证据支持判断。复杂性被放在正确的位置，用户因此获得连续、可控、可复查的云端体验。